



福建理工大学
FUJIAN UNIVERSITY OF TECHNOLOGY

实 验 报 告

实验二 MCU 原理与 ESP 基础外设开发

专 业 交通运输工程

学 号 2251801014

姓 名 罗 丹

日 期 2026 年 6 月 15 日

传感器与检测技术实验报告

实验二：MCU 基础原理

课程名称：传感器与检测技术

实验对象：ESP32-S3 开发板

开发方式：MicroPython + ESP-IDF/PlatformIO 补测

姓 名：罗丹

学 号：2251801014

GitHub: https://github.com/gf-be/ESP32_course

日 期：2026 年 6 月 1 日

福建理工大学

目录

1 摘要	2
2 实验目的与要求	2
3 实验环境与连接方式	3
4 基础外设驱动实验	3
4.1 GPIO 输出与板载 RGB LED	3
4.2 PWM 输出与 ADC 回环采集	4
5 定时器中断与 DMA 双缓冲	5
5.1 查询方式与定时器中断对比	5
5.2 DMA 双缓冲补测	6
6 中断响应延迟与 Jitter 测量	7
6.1 MicroPython 层中断测量	7
6.2 不同中断优先级补测	8
7 时间戳精度测量	9
8 结果分析	10
9 实验任务逐项核对	11
10 提交文件说明	12
11 参考资料	12

1 摘要

本实验基于 ESP32-S3 开发板完成 GPIO 输出、PWM、ADC、定时器中断、GPIO 外部中断、ADC DMA 双缓冲和板端时间戳测量。实验主要采用板内外设回环方式完成验证：GPIO5 输出 PWM 或测试脉冲，GPIO4 用作 ADC 采集输入；ESP-IDF 中断优先级补测中，GPIO7 输出 1 kHz 边沿，GPIO6 作为外部中断输入。

实验结果表明，1 kHz 定时器中断采集的 CPU 占用估计约为 8.494%，明显低于忙等待查询方式；ESP-IDF 连续 ADC DMA 在 50 kHz 目标采样下实测 49997.0 Hz，CPU 占用约为 0.497%；MicroPython 层 10000 次 GPIO 中断响应平均延迟约为 29.723 μs ，jitter 标准差约为 2.241 μs ；ESP-IDF 层 Level 1 与 Level 3 GPIO 中断平均延迟均约为 3.500 μs 。

需要说明的是，标准 MicroPython 固件没有暴露 ESP32 ADC DMA 双缓冲接口，因此 DMA 双缓冲和不同中断优先级对比使用 ESP-IDF/PlatformIO 工程补测完成；MicroPython 部分保留 GPIO、PWM、ADC、定时器、中断和 1 小时时间戳的原始实测数据。

2 实验目的与要求

1. 掌握微控制器 GPIO、PWM、ADC、定时器和外部中断的驱动开发方法。
2. 理解 DMA 双缓冲工作原理，掌握高带宽数据采集的工程实现方式。
3. 掌握嵌入式系统实时性指标测量方法，包括中断响应延迟、jitter 和时间戳精度。
4. 理解实时性对系统的影响，建立微秒级工程精度意识。

3 实验环境与连接方式

表 1: 实验环境与引脚分配

项目	内容
主机系统	Windows
开发语言	MicroPython, ESP-IDF C
开发板	ESP32-S3
板载 LED	GPIO48, NeoPixel RGB
PWM 输出	GPIO5
ADC 输入	GPIO4
中断测试	MicroPython GPIO5 自触发 IRQ; ESP-IDF GPIO7 到 GPIO6 回环
时间戳	time.ticks_ms(), time.ticks_us() 与 PC 时间对比

板内回环连接为 GPIO5 到 GPIO4，用于验证 PWM 输出和 ADC 采集；ESP-IDF 中断优先级补测连接为 GPIO7 到 GPIO6，由 GPIO7 输出 1 kHz 方波，GPIO6 配置为外部中断输入。由于现场没有独立信号发生器，本实验将“外部 1 kHz 方波触发”用同板 GPIO 方波替代，触发路径仍经过 GPIO 输入与中断模块。

4 基础外设驱动实验

4.1 GPIO 输出与板载 RGB LED

ESP32-S3 开发板板载 RGB LED 连接到 GPIO48，使用 NeoPixel 驱动完成输出验证。程序先控制红色 LED 闪烁，再进行蓝色亮度渐变，从而同时验证 GPIO 输出与类 PWM 亮度控制效果。

Listing 1: 板载 RGB LED 输出控制核心代码

```
1 def rgb_set(r, g, b):
2     try:
3         import neopixel
4         np = neopixel.NeoPixel(machine.Pin(RGB_PIN, machine.Pin.OUT), 1)
5         np[0] = (r, g, b)
6         np.write()
7         return True
```

```
8     except Exception as exc:
9         print("# rgb_unavailable,%s" % exc)
10        return False
11
12 def run_board_rgb():
13     ok = rgb_set(0, 0, 0)
14     if not ok:
15         return
16     for _ in range(3):
17         rgb_set(40, 0, 0)
18         time.sleep_ms(200)
19         rgb_set(0, 0, 0)
20         time.sleep_ms(200)
```

运行结果为 GPIO48 板载 RGB LED 完成 3 次红色闪烁，并完成蓝色亮度渐变，说明 GPIO 输出控制正常。

4.2 PWM 输出与 ADC 回环采集

PWM 使用 GPIO5 输出 1 kHz 波形，并回接到 GPIO4 进行 ADC 采集。由于本实验直接采集 PWM 数字波形，没有增加 RC 低通滤波，因此单次 ADC 值主要表现为接近 0 V 或 3.3 V；统计均值和高电平采样比例可以反映占空比变化。

Listing 2: PWM 输出核心代码

```
1 def run_external_pwm():
2     pin = machine.Pin(PWM_PIN, machine.Pin.OUT)
3     pwm = machine.PWM(pin, freq=1000, duty_u16=0)
4     for _ in range(2):
5         for duty in range(0, 65536, 2048):
6             pwm.duty_u16(duty)
7             time.sleep_ms(10)
8         for duty in range(65535, -1, -2048):
9             pwm.duty_u16(duty)
10            time.sleep_ms(10)
11    pwm.deinit()
12    pin.off()
```

表 2: PWM 到 ADC 回环统计结果

PWM 占空比	样本数	ADC 均值	估算电压	高电平采样比例
0%	1500	0.00	0.0000 V	0.00%
25%	1500	16122.35	0.8118 V	24.60%
50%	1500	32680.82	1.6456 V	49.87%
75%	1500	48932.94	2.4640 V	74.67%
100%	1500	65535.00	3.3000 V	100.00%

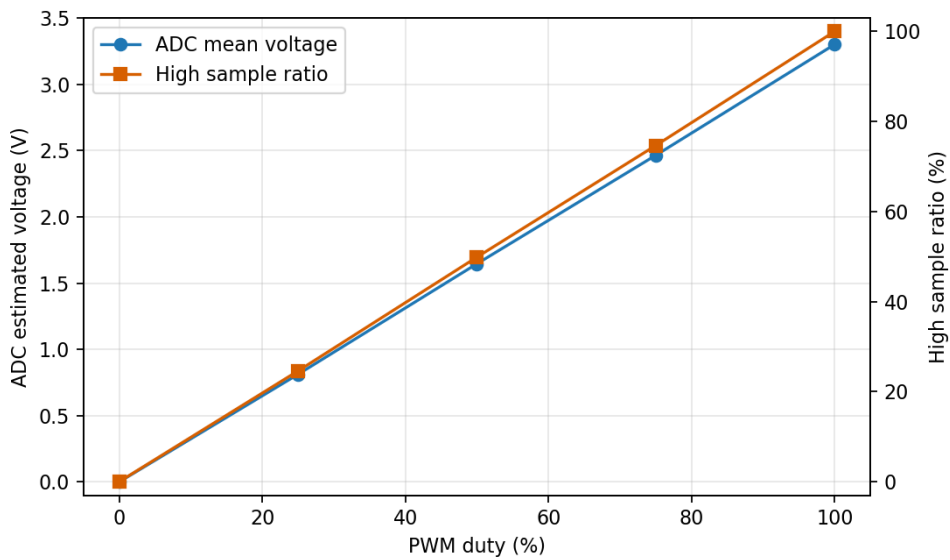


图 1: PWM 占空比与 ADC 采样统计

从图表可见，占空比 25%、50%、75% 时，高电平采样比例分别约为 24.60%、49.87%、74.67%，与设定值基本一致，说明 PWM 输出与 ADC 采集链路有效。

5 定时器中断与 DMA 双缓冲

5.1 查询方式与定时器中断对比

MicroPython 测试脚本 `code/02_polling_vs_timer_adc.py` 对比了最大速率查询方式、1 kHz 忙等待查询方式和 1 kHz 定时器中断方式。定时器中断方式在回调函数中读取 ADC，并记录相邻采样时间间隔和服务时间。

Listing 3: 定时器中断 ADC 采样核心代码

```
1 def timer_cb(timer):
2     global timer_count, timer_last
3     t0 = time.ticks_us()
4     if timer_count == 0:
5         timer_last = t0
6     if timer_count < TIMER_N:
7         timer_values[timer_count] = adc_read()
8         timer_dt[timer_count] = time.ticks_diff(t0, timer_last)
9         timer_last = t0
10        timer_service[timer_count] = time.ticks_diff(time.ticks_us(), t0)
11        timer_count += 1
```

5.2 DMA 双缓冲补测

ESP-IDF 补测工程位于 code/espidf_dma_irq/。程序使用 ADC continuous driver，从硬件 DMA 环形缓冲区批量取样，并按帧统计吞吐量、处理时间和 CPU 占用。由于 MicroPython 标准固件未暴露 ESP32 ADC DMA 双缓冲接口，DMA 部分使用 ESP-IDF C 工程实现更符合底层外设能力。

表 3: 采集方式性能对比

采集方式	最大或目标采样率	CPU 占用率	丢包率	适用场景
查询方式	13586.7 Hz	100.0%	0.000%	极简单低速采样或极限轮询测试
1 kHz 忙等待查询	1000.3 Hz	100.0%	0.000%	需要固定频率但不能释放 CPU 的临时测试
定时器中断	999.0 Hz	8.494%	0.000%	周期稳定、CPU 可执行其他任务的采集
ESP-IDF 查询方式	18373.7 Hz	100.000%	0.000%	C 层阻塞轮询基准
DMA 双缓冲	49997.0 Hz	0.497%	0.000%	ESP-IDF 连续 ADC DMA，高带宽批量采集

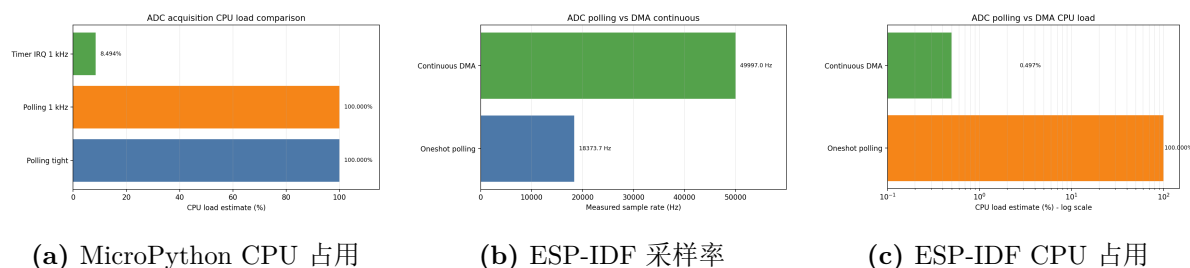


图 2: ADC 采集方式对比

结果显示，查询方式虽然实现简单，但 CPU 占用接近 100%；定时器中断可以在 1 kHz 下显著释放 CPU；DMA 双缓冲进一步将稳定采样率提高到约 50 kHz，同时将 CPU 占用降低到 0.497%。这体现了 DMA 在高带宽数据采集中的核心优势：数据搬运由硬件完成，CPU 只需按缓冲区批量处理。

6 中断响应延迟与 Jitter 测量

6.1 MicroPython 层中断测量

实验先使用 MicroPython 在 GPIO5 输出 1 kHz 周期边沿，并在同一 GPIO 上配置上升沿 IRQ，记录 10000 次响应延迟和相邻 IRQ 间隔。

Listing 4: GPIO 中断响应测量核心代码

```

1 def irq_cb(p):
2     global irq_count, last_irq_us
3     if not armed:
4         return
5     idx = irq_count
6     if idx < N:
7         now = time.ticks_us()
8         latencies[idx] = time.ticks_diff(now, current_edge_us)
9         if idx == 0:
10             intervals[idx] = 0
11         else:
12             intervals[idx] = time.ticks_diff(now, last_irq_us)
13         last_irq_us = now
14         irq_count = idx + 1

```

表 4: MicroPython 层 10000 次中断响应统计

指标	数值
样本数	10000
最小响应延迟	29.000 μs
最大响应延迟	249.000 μs
平均响应延迟	29.723 μs
jitter 标准差	2.241 μs
丢包率	0.000%
IRQ 间隔均值	999.972 μs
IRQ 间隔 jitter	6.406 μs

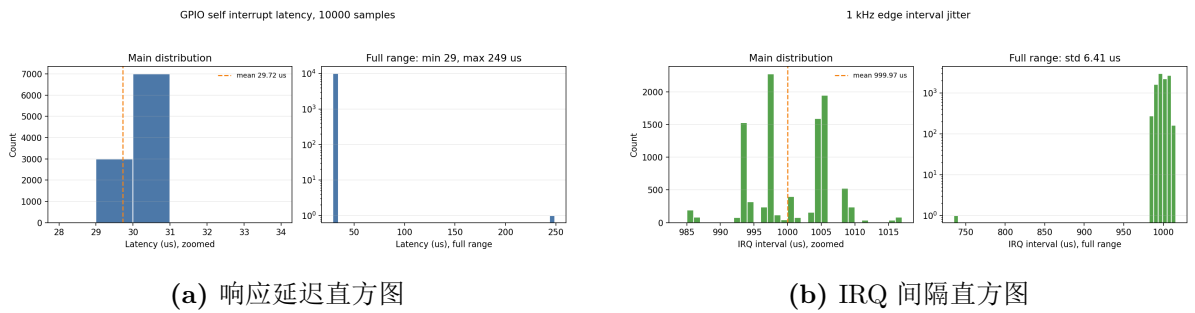


图 3: MicroPython 中断响应统计分布

绝大多数响应延迟集中在约 30 μs 附近。MicroPython 结果包含解释器、回调和 `ticks_us()` 读取开销，因此它反映的是“MicroPython 层可见响应延迟”，不是裸机 C 或硬件中断入口延迟。

6.2 不同中断优先级补测

ESP-IDF 补测使用 GPIO7 到 GPIO6 回环，分别配置 Level 1 和 Level 3 中断优先级，比较不同优先级下的响应延迟差异。

表 5: ESP-IDF 不同中断优先级响应结果

优先级	样本数	丢包率	最小延迟	最大延迟	平均延迟	jitter	IRQ 间隔均值
Level 1	10000	0.000%	3.000 μs	4.000 μs	3.500 μs	0.500 μs	1000.000 μs
Level 3	10000	0.000%	3.000 μs	4.000 μs	3.500 μs	0.500 μs	1000.000 μs

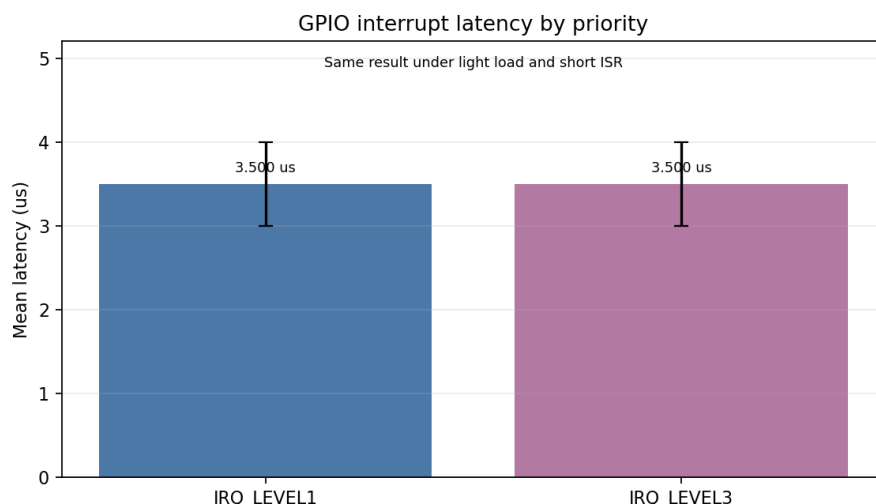


图 4: ESP-IDF 不同中断优先级延迟对比

在本次测试负载较低、ISR 很短、没有额外高优先级任务抢占的条件下，Level 1 与 Level 3 的平均延迟均为 $3.500\ \mu\text{s}$ ，差异不明显。中断优先级的影响通常需要在存在竞争中断、长时间临界区或高优先级任务抢占时才更容易体现。

7 时间戳精度测量

时间戳测量脚本每秒输出一组 ESP32 `ticks_ms()` 和 `ticks_us()`，PC 端捕获脚本同步记录 `time.time_ns()`，再计算 ESP32 相对 PC 标准时间的漂移。

Listing 5: 板端时间戳输出核心代码

```

1 PERIOD_MS = 1000
2 DEFAULT_SAMPLES = 3600
3
4 freq = machine.freq()
5 start = time.ticks_ms()
6 next_t = start
7 for i in range(DEFAULT_SAMPLES + 1):
8     while time.ticks_diff(time.ticks_ms(), next_t) < 0:
9         time.sleep_ms(5)
10    print("%d,%d,%d,%d" % (i, time.ticks_ms(), time.ticks_us(), freq))
11    next_t = time.ticks_add(next_t, PERIOD_MS)

```

本次采集共获得 3601 条板端时间戳记录，PC 参考时长 3599.840 s，1 小时末端偏差约 158.536 ms，折算晶振或计时链路频率误差约 44.040 ppm。ppm 计算方式为：

$$\text{ppm} = \frac{T_{\text{ESP32}} - T_{\text{PC}}}{T_{\text{PC}}} \times 10^6$$

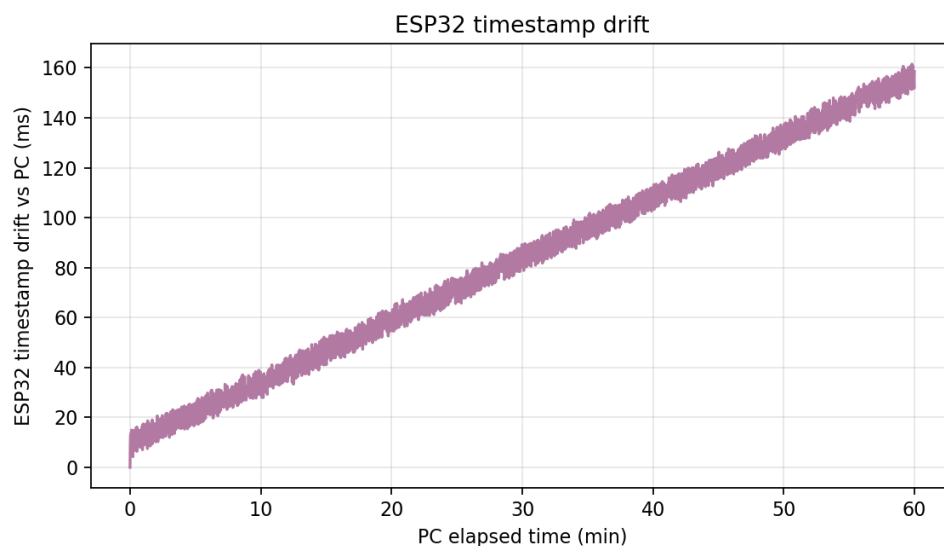


图 5: ESP32 时间戳相对 PC 时间漂移曲线

PC 时间并不是实验室级标准时钟,因此本实验 ppm 反映的是 ESP32 MicroPython 时间戳链路相对 PC 系统时钟的偏差,包含串口输出、PC 调度和系统时钟误差。若需要更严格结果,应使用 GPSDO、频率计或逻辑分析仪作为基准。

8 结果分析

1. GPIO 和 PWM 可直接由 ESP32-S3 板内外设完成。GPIO48 板载 RGB LED 能直观验证输出功能,GPIO5 PWM 回接 GPIO4 后可通过 ADC 统计占空比。
2. 查询方式实现简单,但在高频采样时 CPU 占用接近 100%,不适合复杂实时任务。
3. 定时器中断方式能稳定实现 1 kHz 周期采样, CPU 占用明显低于忙等待查询方式,是中低速周期采集的合理方案。
4. 中断响应延迟的 jitter 来自解释器调度、回调执行时间、`ticks_us()` 时间戳读取开销和串口或系统后台任务影响。MicroPython 层测得约几十微秒量级,适合课程级实时性认识;ESP-IDF 裸机 ISR 补测可把可见延迟降低到约 $3.5 \mu\text{s}$ 。
5. DMA 双缓冲适用于更高带宽 ADC 采集。其核心优势是 ADC 数据搬运由 DMA 完成, CPU 只在缓冲区事件中批量处理数据,从而降低中断频率和 CPU 负载。

6. 在低负载、短 ISR 条件下，Level 1 与 Level 3 中断优先级延迟结果相同，说明优先级差异主要在中断竞争、长临界区或高优先级任务抢占时体现。
7. 时间戳漂移 ppm 可用于理解晶振误差对同步系统的影响。例如 100 ppm 误差会造成每秒 100 μ s、每小时约 360 ms 的累积偏差，因此长时间同步需要校时或频率补偿。

9 实验任务逐项核对

表 6: 实验任务完成情况核对

实验要求	完成情况	证据文件或结果
GPIO 输出控制 LED	已完成	GPIO48 板载 RGB 闪烁， code/01_gpio_pwm_adc_timer_demo.py
PWM 呼吸灯或输出	已完成	GPIO48 亮度渐变，GPIO5 PWM 输出
ADC 采集	已完成	data/pwm_adc_loopback_stats.csv， GPIO5 到 GPIO4 板内回环
1 kHz 定时器中断采集	已完成	data/polling_vs_timer_adc_smoke.csv
查询与中断 CPU 占用对比	已完成	查询 100%，定时器约 8.494%
DMA 双缓冲 ADC	已完成	ESP-IDF 连续 ADC DMA：49997.0 Hz， CPU 0.497%
10000 次中断响应统计	已完成	data/self_irq_latency_10000.csv，平均 29.723 μ s
时间戳精度测量	已完成	1 小时时间戳数据与漂移曲线
不同中断优先级对比（选做）	已完成	Level 1 与 Level 3 各 10000 次，平均延迟均为 3.500 μ s

10 提交文件说明

表 7: 提交包文件说明

文件或目录	内容
report.md	原始 Markdown 实验报告
index.html	可直接浏览的网页报告
data/	原始数据、清洗数据和统计 JSON
assets/	PWM-ADC、CPU 占用、DMA 采样率、中断延迟、优先级对比、时间漂移图
code/	MicroPython 板端脚本和 ESP-IDF DMA/IRQ 补测工程
README.md	提交包说明

11 参考资料

[1] MicroPython ESP32 Quick Reference: <https://docs.micropython.org/en/latest/esp32/quickref.html>

[2] MicroPython machine 模块文档: <https://docs.micropython.org/en/latest/library/machine.html>

[3] Espressif ESP-IDF ADC Continuous Mode Driver Programming Guide: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/adc_continuous.html